

**Regex**

- ^ start of line
- \$ end of line
- < , > start/end word
- . any one character
- [abc] [a-z] [A-Z]
- ? 0 or more occ. of prev.
- \* 1 or more ? 0 or one
- n {min, max}
- | or ( ) \: escape
- ed 3 - global i - case insens.
- sed "s/regex/replacement/flags" find repl.
- sed "y/character/replacement/" translit.
- sed "r/regex/d" delete lines that match

**grep**

- l quiet
- c no. delin.
- v no match
- n no. lines in file
- r recursive
- i case insensitive
- o only part that matches
- w only matches 1 word
- b word boundary
- E
- sed -n /pattern/p > grep

open(file, flags, mode) => fd  
(if creating file)  
read(fd, &var, size)  
write(fd, &var, size)  
close(fd) lseek(fd, offset, whence)

offset = where I am in specific page/segment

paging => pages of fixed size => bitli din stg = VPN

x bytes = 2<sup>4</sup> => y bits in total => offset no bits = same but with page size  
=> VPN: total - offset bits

**THREADS** <pthread.h>

pthread\_create(&threadID, NULL, void\*(function)(void\*), void\*arg)

pthread\_join(threadID, void\*status)

pthread\_mutex\_t my\_mutex

pthread\_mutex\_lock(&my\_mutex)

pthread\_mutex\_unlock(&my\_mutex)

pthread\_cond\_t cond; pthread\_cond\_wait(&cond, &mutex)

pthread\_cond\_signal(&cond)

pthread\_barrier\_t barrier; pthread\_barrier\_wait(&barrier)

pthread\_rwlock\_t rwlock; pthread\_rwlock\_rdlock(&rwlock)

pthread\_rwlock\_wrlock(&rwlock)

pthread\_rwlock\_unlock(&rwlock)

sem\_t sem; sem\_init(&sem, 0, value)

sem\_wait(&sem) => decrem val by 1 wait if < 0

sem\_post(&sem) => increm. by 1 wake up 1 sleeping thread

binary semaphore => locks (value = 1)

pthread\_mutex\_t mu = PTHREAD\_MUTEX\_INITIALIZER

pthread\_mutex\_init(&mu, NULL)

pthread\_mutex\_destroy(&mu)

pthread\_mutex\_lock(&mu)

pthread\_mutex\_unlock(&mu)

pthread\_cond\_t cond = PTHREAD\_COND\_INITIALIZER

pthread\_cond\_init(&cond, NULL)

pthread\_cond\_destroy(&cond)

pthread\_barrier\_t barrier = PTHREAD\_BARRIER\_INITIALIZER

pthread\_barrier\_init(&barrier, NULL, count)

pthread\_barrier\_destroy(&barrier)

pthread\_rwlock\_t rwlock = PTHREAD\_RWLOCK\_INITIALIZER

pthread\_rwlock\_init(&rwlock, NULL)

pthread\_rwlock\_destroy(&rwlock)

pthread\_rwlock\_rdlock(&rwlock)

pthread\_rwlock\_wrlock(&rwlock)

pthread\_rwlock\_unlock(&rwlock)

pthread\_rwlockattr\_t attr = PTHREAD\_RWLOCKATTR\_INITIALIZER

pthread\_rwlockattr\_init(&attr, NULL)

pthread\_rwlockattr\_destroy(&attr)

pthread\_rwlockattr\_setpshared(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

pthread\_rwlockattr\_setscope(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

pthread\_rwlockattr\_setscope(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

pthread\_rwlockattr\_setscope(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

pthread\_rwlockattr\_setscope(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

pthread\_rwlockattr\_setscope(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

pthread\_rwlockattr\_setscope(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

pthread\_rwlockattr\_setscope(&attr, 1)

pthread\_rwlockattr\_setkind\_np(&attr, 1)

pthread\_rwlockattr\_setrobust(&attr, 1)

|      |                   |                 |                        |
|------|-------------------|-----------------|------------------------|
| byte | 1 byte            | 2 <sup>0</sup>  | 2 <sup>0</sup> x 1024  |
| KB   | 1024              | 2 <sup>10</sup> | 2 <sup>10</sup> x 1024 |
| MB   | 1024 <sup>2</sup> | 2 <sup>20</sup> | 2 <sup>20</sup> x 1024 |
| GB   | 1024 <sup>3</sup> | 2 <sup>30</sup> | 2 <sup>30</sup> x 1024 |
| TB   | 1024 <sup>4</sup> | 2 <sup>40</sup> | 2 <sup>40</sup> x 1024 |

non deadlock bugs => code doesn't execute in desired sequence

atomicity viol.: code block assumes a shared resource won't change but a thread changes it

order viol.: Thread A assumes B has already executed

4 Deadlock Cond => deadlock occurs when all 4 met & mult

if break one => solve

\* mutual exclusion: only one thread can hold a resource at time (NOT shared)

\* hold-and-wait: thread holds resource while waiting for another one

\* no preemption: OS cannot forcibly take resource from thread

\* circular wait: a chain of threads where each waits for a resource held by next in chain (T1 -> T2 -> T3 -> T1)

\* trylock() instead of lock()

or livelock -> release all locks

\* always lock in the same order

\* wrap all locks in master lock

\* Valgrind that don't use locks at all

**SCHEDULING**

\* RoundRobin: each process executes unpic in a given time

\* turnaround time = T<sub>complet</sub> - T<sub>arrival</sub>

\* response time = T<sub>first-run</sub> - T<sub>arrival</sub>

\* Multilevel Feedback Queue (MLFQ): Priority A > Priority B => A runs in cept de la highest priority => coobara after some B => back to A

**awk** awk /pattern/ {action}

\$0 = entire line \$1... = fields

NR = no. fields NR = current line

FS = separator (F) change sep.

getline /n matches/not pattern

shell #!/bin/bash cpmd >> file.sh

find dir -type f -name "\*.c" (file in dir)

var=" " \$var \$( ) run cmd > output in var

\$0 = name of command \$1... = line args

@ or \$\* = all args \$# = count args

flags: -f =? file -d =? dir -e =? e

-r, -w, -x =? permis -s = empty?

=, !, -, -z (empty), -m (nonempty)

if [ I; then ... fi

while; do... done

for do... done

EL happy trail II fail trail; regardless

PROCESS = program in execution

\* race cond = proc. access same resource

fork() = dupli., everything 2<sup>nd</sup> proc.

parent => child id child => 0

wait() waitpid() = wait for children

exit() = proc. termin

signal: man 7 signal

exec(): replace code without p. /bin...

execvp() execvp() execvp(argv[0], argv+1)

pipe(p) p[0] read p[1] write

close(p[0])

int mkfifo(path, mode) open(026, 0, WRONLY)

head -n (-p = all but last)

tail -n (-f = following from line 0)

cpmd > file 2> 1

cpmd > file

0 - read 1 - write

sem\_wait() => decrem val by 1 wait if < 0

sem\_post() => increm. by 1 wake up 1 sleeping thread

binary semaphore => locks (value = 1)

FILE SYSTEM

\* access: open, read, write

\* disk divided into blocks

\* user data = data region

\* metadata: size of file, owner, access rights

\* inode (i-number): block = (number \* size of (inode)/block size)

sector = (block \* block.size + inode.addr) / sectorsize

\* len, size, protection

\* direct ptr -> one disk + double ptr., triple ptr.

\* open: FS reads the inode -> inumber -> find ptr. to data blocks -> traverse path

\* FAT - simple linked-based allocation scheme

TLB - translation lookaside buffer (fast fetch data from mem.)

if TLB miss -> MMU go to Page Table (in RAM) -> translation

-> MMU has physical addr.

-> CPU checks cache -> hit -> read (fast)

-> miss -> retrieve from RAM

if not on RAM (page fault present bit = 0) -> find on disk

\* allocation structs: track if inode free/alloc.

\* tracking methods SLL / bit map (data + inode)

\* superblock (info about file sys.)

disk = superblock + bit map + inode table + data blocks

\* db + inodes are free to use

\* content of files

\* fetch data

\* from memory

MMU = T<sub>mem</sub> / T<sub>phys</sub>

avg mem access time

T<sub>mem</sub> = cost of mem. access

T<sub>phys</sub> = cost of phys. access

phys. addr = base + virtual

if not on RAM (page fault present bit = 0) -> find on disk

\* starvation -> process postponed to infinity bc of scheduler

-> SJF (shortest job 1st): job with sh. exec time ✓ min avg time

-> SFCF (sh. time to completion): preemptive SJF stop work if shorter arrives

-> Lottery scheduling: pick random ticket

-> Multilevel Feedback Queue (MLFQ): Priority A > Priority B => A runs in cept de la highest priority => coobara after some B => back to A



restricted operations → user mode (lim. access)

OS runs in kernel mode

regain control over CPU → sys calls, illegal op.

context switch → OS saves state of running process and restore state of another one

→ addr. space = abstraction of phys. mem. → code, stack, heap,

→ virtual mem. = technique → OS gives each proc. addr. space + uses hardware to translate

virtual addr. into phys. mem (RAM)

memory management

→ free list, trap table → remember addr. of the sys call handler

segmentation → div. in logical segm. code stack heap

- segm. fault → access memory that is not allowed

- external fragm. → create multiple small gaps

→ paging → div. into fixed segm.

policies → which page to evict (Replacement Policies)

• FIFO → oldest page evicted regardless of how often it is accessed

• LRU (Least Recently Used) → evicts the page that hasn't been accessed for longest time

• LFU (Least Frequently Used) → evicts page with lowest reference count

- MFU (Most Frequently Used) → (used enough now mai trebe) → page with highest reference count

- MRU (Most Recently Used) → (used recently wont need soon) → page that was most rec. accessed

• Random

Process States:

- running → actively executing instructions on CPU
- ready → ready to run but waits for scheduler
- blocked (waiting) → cannot run because waiting for event (I/O from disk)

Disk & RAID (Redundant Array of Inexp. Disks) → slow comp. to RAM

disk: data stored in blocks → to get data → OS requests → seek (move arm) + waiting to spin

RAID: combine multiple cheap disks into one high perform.

RAID 0: striping: if one disk dies → no lost everything

RAID 1: mirroring: 50% capacity (copy on 2 disks everything)

RAID 4: striping + parity (xor to get parity)

Hard link → same file but on disk (directory that points to file inode)

→ delete: data remains accessible via any remaining hard links (only deleted when link count = 0)

Symbolic link → file (special) that contains path to another file

→ delete: delete target file → broken/dangling link, unaffected target

→ can create across diff. FS + link to directories

ssh username@server.ip